ARCHITECTURAL NOTE

L4 Glitch Map

Concept Draft v0.1

A formatted presentation of the concept draft focused on reality-bound scenario diagnostics, L4 boundary visibility, and bounded continuation logic.

> Where does the branch break under L4?

AUTHOR	Ivan Kotov
DATE	2026-04-01
STATUS	Exploratory architectural note
LANGUAGE	EN

Bruxelles - 2026

1. Executive Summary

A recurring weakness in current AI discourse is that failure is usually described after the fact, in language, logs, or policy review. This is too late.

A long-lived system operating under real constraints should be able to expose the point at which a scenario stops being viable **before** unchecked execution continues.

L4 Glitch Map is a proposed visual and architectural layer for representing where a scenario branch collides with reality.

It does not ask only whether a path is elegant, persuasive, or computationally reachable.

It asks a stricter question:

Where does the branch break under L4?

That means under:

- energy cost,
- time windows,
- thermal and hardware limits,
- maintenance burden,
- privilege boundaries,
- trust surfaces,
- evidence requirements,
- and continuity constraints.

The purpose of L4 Glitch Map is not cinematic effect. Its purpose is to make the collision with reality visible, classifiable, and actionable.

A branch that fails should not disappear into vague language such as “not feasible” or “needs improvement.” It should leave behind a structured failure object that can:

- constrain execution,
- narrow authority,
- request evidence,
- permit or deny a research fork,
- and preserve the reason for failure across time.

In that sense, L4 Glitch Map is not a visualization gimmick. It is a **reality-diagnostic layer** for long-lived AI entities and related scenario systems.

---

2. Problem Statement

Most AI systems today expose one of two bad habits:

1. **Silent optimism**
   A scenario continues because no explicit boundary was modeled.
2. **Late refusal**
   The scenario is allowed to accumulate momentum and only later receives a textual rejection, policy block, or audit objection.

Both are weak.

A serious architecture should expose the exact place where a path ceases to remain coherent under real-world conditions.

This matters especially for:

- persistent entities (`c`) operating under L4,
- agentic systems with bounded privileges,
- DEA / EA-L4 style transitions from input to experience,
- witness-backed scenario analysis,
- home robotics,
- ocean / remote autonomy,
- and speculative engineering where fantasy and reality must remain separated without losing the value of either.

---

3. Core Thesis

A scenario does not become invalid only because a human says “no.” A scenario becomes invalid when it crosses a boundary that the system cannot carry without degradation, illegitimacy, incoherence, or unbounded risk.

L4 Glitch Map is the layer that marks this crossing.

A glitch is therefore not merely an error message. It is the architectural trace of a failed continuation under constraint.

A mature system should be able to say, structurally:

- this branch remains valid,
- this branch is degraded but still bounded,
- this branch is speculative and quarantined,
- this branch is blocked by a hard L4 lock,
- this branch may continue only as a research node,

• this branch invalidates continuity or authority if pursued.

4. Explicit Bridge

EXPLICIT BRIDGE:

```
scenario exploration -> L4 boundary detection -> bounded accountability
```

This is the direct bridge between visual branching logic and reality-bound governance.

A scenario branch is not valuable because it is imaginative. It becomes valuable when the system can determine whether it remains:

- executable,
- researchable,
- witnessable,
- or quarantined.

5. Hidden Bridge A — Experience Formation

NOT EVERY FAILED PATH IS USELESS.

A failed path can become an **Experience Artifact precursor** if the failure is carried forward as structured consequence rather than discarded as narrative residue.

That means the glitch point can feed:

- constraint learning,
- uncertainty markers,
- authority narrowing,
- and later training-origin hygiene.

This creates a quiet bridge from scenario failure to DEA / EA-L4 style experience formation.

A glitch is therefore not only a stop sign. It is also a possible seed of bounded experience.

6. Hidden Bridge B — Witness and Recognition

A GLITCH THAT CHANGES PRIVILEGES, NARROWS SCOPE, OR BLOCKS ACTION CANNOT REMAIN PURELY LOCAL INTUITION.

For high-impact paths, the system should be able to emit a witness-bearing event that states:

- where the branch failed,
- what type of lock was triggered,
- what evidence was missing,
- whether a speculative bridge was allowed,
- and who remained accountable for the transition.

This forms a quiet bridge toward witness logic, continuity preservation, and recognition across systems.

In other words:

a glitch is not only a visual event; it can become an auditable event.

7. Earth Paragraph

In engineering and anatomy, failure rarely arrives as an abstract argument. A bridge fails because load exceeds tolerance. A cooling loop fails because heat is not removed fast enough. A joint fails because repeated stress outruns recovery. A body fails because compensation cannot continue forever.

Serious systems survive by exposing strain early: cracks, heat, pressure, fatigue, latency, oxygen debt, loss of blood supply, or control oscillation.

L4 Glitch Map applies the same principle to long-lived AI architectures: it marks where a path stops being sustainable before the system confuses possibility with viability.

8. Glitch vs Error vs Refusal

These terms should not be collapsed.

8.1 Error

A local malfunction, inconsistency, parsing issue, or implementation defect.

8.2 Refusal

A policy or privilege outcome stating that an action should not proceed.

8.3 Glitch

A structural marker showing that a scenario branch has collided with a modeled reality boundary.

A glitch may lead to:

- refusal,

- research fork,
- witness event,
- narrowed execution,
- degraded continuation,
- or hard stop.

But the glitch itself is the diagnostic object.

---

9. Lock Taxonomy

The system should classify glitches into explicit lock types.

9.1 Energy Lock

The branch exceeds available energy budget, power envelope, or metabolic allowance.

9.2 Time Lock

The branch cannot complete inside the valid time window, deadline, cron phase, or recovery horizon.

9.3 Thermal Lock

The branch is invalid under cooling, sustained load, or hardware thermal safety conditions.

9.4 Maintenance Lock

The branch depends on replacement, calibration, repair, supply chain, or human upkeep beyond permissible bounds.

9.5 Privilege Lock

The branch requires authority or capability not currently admissible under least privilege and challengeability constraints.

9.6 Trust Lock

The branch depends on unverifiable, untrusted, unanchored, or spoofable external inputs.

9.7 Evidence Lock

The branch cannot continue because required witness, provenance, measurement, or validation artifacts are missing.

9.8 Continuity Lock

The branch would damage the persistent identity, accountable center, or continuity conditions of `C`.

9.9 Embodiment Lock

The branch assumes physical action, manipulation, movement, or intervention that the current embodiment cannot carry safely.

9.10 Legibility Lock

The branch may proceed computationally, but the result would become non-auditable, non-reconstructable, or opaque beyond acceptable limits.

---

10. Minimal Object Model

A glitch should exist as a first-class architectural object.

```
GlitchNode:
  glitch_id: string
  scenario_path: string
  parent_node_id: string
  lock_type: enum
  severity: enum               # advisory | narrowing | blocking | terminal
  failed_constraint: string
  local_explanation: string
  evidence_required: list
  privilege_effect: enum       # none | narrow | challenge | deny
  continuity_effect: enum      # none | stress | degrade | invalidate
  allowed_bridge_types: list   # research | simulation | quarantine | none
  witness_required: bool
  emitted_at: timestamp
  responsible_anchor: string
```

This object is intentionally small. It should be easy to emit, visualize, hash, and carry forward.

---

11. State Outcomes

When a glitch is triggered, the system should not default to one reaction. It should resolve into one of several bounded outcomes.

11.1 Green — Valid Continuation

The branch remains within L4 constraints.

11.2 Yellow — Constrained Continuation

The branch may continue, but under narrowed privileges, shorter scope, or additional witness burden.

11.3 Amber — Research Fork

The branch is not executable as-is, but may continue as a quarantined research node.

11.4 Red — Hard Lock

The branch is blocked. No execution path may proceed without new evidence or architecture change.

11.5 Grey — Unresolved / Quarantined

The system cannot classify the branch adequately; continuation is suspended pending human review, evidence, or decomposition.

---

12. UI Semantics

The visual layer should remain subordinate to the diagnostic logic.

Possible display semantics:

- **glitch fracture** = hard L4 collision,
- **heat bloom** = energy/thermal stress,
- **latency fog** = time-window degradation,
- **narrowing ring** = privilege contraction,
- **witness beacon** = auditable event emitted,
- **quarantine halo** = speculative bridge allowed but not legitimized,
- **dead branch fade** = continuity-unsafe path terminated.

Important rule: visual semantics must never make a speculative branch look equivalent to a validated branch.

---

13. Speculative Bridges

A useful glitch layer should not confuse all failure with deletion.

Some blocked paths may be allowed to continue as bounded speculative bridges. That continuation must remain explicitly marked as one of the following:

- **Research Bridge** — hypothesis requiring validation,
- **Simulation Bridge** — local model continuation without real-world authority,
- **Quarantine Bridge** — isolated branch prevented from contaminating executable paths,
- **Pedagogical Bridge** — explanation or learning branch with no authority effect.

This separation is crucial. Otherwise, fantasy contaminates authority.

---

14. Relation to `c = a + b`

In a `c = a + b` architecture, the glitch does not belong primarily to the model. It belongs to the path by which `c` attempts to remain coherent under L4.

That means:

- continuity can be stressed,

- authority can be narrowed,
- witness demands can increase,
- execution can fail closed,
- and some speculative branches may remain visible without being granted legitimacy.

The key distinction is simple:

a reactive system can ignore the cost of a broken branch; a persistent system must carry it.

---

15. Relation to DEA / EA-L4 / Witness

15.1 DEA

A glitch may indicate that an input changed system interpretation but failed to become valid experience under continuity and constraint.

15.2 EA-L4 / EATP

A glitch can become a precursor to an Experience Artifact only if the failed path is transformed into bounded, consequence-carrying structure.

15.3 L4 Witness

High-impact glitches should emit witness-capable records when they affect action authority, evidence demands, or continuity state.

Thus the chain becomes clearer:

```
input -> path -> glitch -> branch classification -> experience / quarantine / witness
```

---

16. Example A — Home Robot Boundary

Scenario branch: A household robot proposes autonomous pantry reorganization during the night.

Glitch: The branch crosses a combined privilege + presence boundary. The robot has movement capability but not standing permission for nocturnal rearrangement in shared private space.

Outcome:

- Privilege Lock triggered
- branch narrowed to suggestion-only mode
- witness not required
- speculative bridge not needed
- continuation allowed only after explicit daytime authorization

This is not a software bug. It is an L4 glitch in domestic coexistence.

17. Example B — Ocean Autonomy Boundary

Scenario branch: A remote seabed entity proposes prolonged independent exploration after communications degrade.

Glitch: Current energy reserves, maintenance uncertainty, and vector-of-return constraints make open-ended continuation non-viable.

Outcome:

- Energy Lock
- Maintenance Lock
- Continuity stress
- research fork may remain for future architecture work
- live mission path collapses into return-or-suspend logic

The point is not that the system was “wrong.” The point is that the branch is not sustainably inhabitable under present L4 conditions.

18. Example C — Training-Origin Boundary

Scenario branch: A system proposes converting a scenario walkthrough directly into training material.

Glitch: The branch lacks witness, consequence-bearing validation, and clear separation between speculative and reality-tested paths.

Outcome:

- Evidence Lock
- Legibility Lock
- branch moves to quarantine
- no authority is granted to downstream learning ingestion

This prevents polished fiction from quietly entering the training lineage as if it were experience.

19. Why This Matters

Without an explicit glitch layer, architectures tend to fail in one of two unhealthy directions:

- they become overconfident and continue too far,
- or they reduce everything to generic refusal language.

Neither is good enough for long-lived systems.

A mature architecture should preserve:

- where the path broke,
- why it broke,
- what would be required to reopen it,
- what kind of continuation remains legitimate,
- and whether the accountable center of the system remained intact.

---

20. Open Questions

1. Which lock classes should be mandatory in a minimal public implementation?
2. Which glitches should always emit witness events?
3. How should quarantine branches be rendered without seducing users into mistaking them for validated paths?
4. Can glitch density become a health metric for an entity or architecture?
5. How should glitch-derived research nodes be fed back into DEA / EA-L4 without polluting legitimacy?

---

21. Conclusion

L4 Glitch Map is a proposal for making reality collision explicit.

Its purpose is not to dramatize failure. Its purpose is to stop architectures from confusing imagination, possibility, and survivable continuation.

A branch that breaks should remain visible. But it should remain visible in the correct category.

That is the beginning of a better discipline: not merely generating paths, but learning to classify where reality accepts them, where it narrows them, and where it refuses to carry them further.

The future of intelligent systems will not be shaped only by what they can generate. It will also be shaped by how clearly they can expose the points where the world says:

not like this.